

BuildNexus

Sprint 1 Quality Assurance Verification Report

PROJECT BuildNexus (Microservices)	SPRINT CYCLE Sprint 1	REPORT DATE 2026-09-01	PREPARED BY QA- IT24100312

1. EXECUTIVE SUMMARY

Metric / Item	Result	Details / Notes
Stories in Scope	10	100% acceptance criteria covered
Stories QA-Passed	10	All user stories fully verified
Stories Failed	0	No stories rejected
Blocking Defects	0	Zero critical/blocking issues open
Non-Blocking Observations	4	Tracked in Section 7 (tooling & dev configs)
Backend Automated Tests (Local Executed)	405 / 446 Passed	405 passed; 41 skipped locally (require Docker DB)
Backend Automated Tests (CI Pipeline)	446 / 446 Passed	100% pass rate in GitHub Actions CI pipeline
Frontend Component Tests	Pass in CI	7 test suites passing in CI (Node 22)
Overall QA Recommendation	PASS — Recommended for Sprint 1 Sign-Off	

Sprint 1 delivers the platform's core identity spine (registration, login, JWT issuance, RBAC, password reset, profile self-service, and admin user management) alongside the initial construction project slice (create project, view/update status, event integration). All services operate behind a single YARP API Gateway with automated CI pipelines enforcing unit and integration quality gates on every push.

2. SCOPE — STORIES UNDER TEST

Story	Jira ID	Title / Description	Target Components
US-01	SCRUM-14	User Registration & Login	user-service, frontend, gateway
US-02	SCRUM-15	Manage User Profile	user-service, frontend
US-03	SCRUM-16	Enforce Role-Based Access Control	user-service, frontend, gateway
US-04	SCRUM-17	Password Reset	user-service, frontend, Mailpit
US-05	SCRUM-18	Create Construction Project	project-service, frontend
US-06	SCRUM-19	View & Update Project Status	project-service, frontend
US-22	SCRUM-35	Project Event Integration (outbox → Kafka)	project-service, Kafka
US-25	SCRUM-38	API Gateway Routing & Integration	api-gateway
US-35	SCRUM-41	Configure CI/CD Pipeline	.github/workflows/ci.yml
US-37	SCRUM-50	Admin: Manage Users & Roles	user-service, frontend

Out of Scope for Sprint 1 (Explicitly Excluded)

- **Design & Document Management Service (US-09):** Service not implemented; /api/designs route reserved at Gateway level.
- **Construction & Payment Services:** Basic project scaffolding only; logic scheduled for future sprints.
- **Deployment & Docker-Build CI Stages:** Scheduled for Sprint 3-4; intentionally not stubbed in pipeline.
- **Bulk Admin Actions & Export:** CSV export and email notifications on role changes deferred to future backlog.

3. TEST ENVIRONMENT SPECIFICATION

Component Layer	Configuration Details
API Gateway	http://localhost:5000 (YARP) – Single entry point for all service traffic
User Service	http://localhost:5001 MySQL Database: buildnexus_user_db
Project Service	http://localhost:5002 MySQL Database: buildnexus_project_db (Port 3307)
Email Service	Mailpit SMTP mailpit:1025 Web GUI: http://localhost:8025
Event Bus	Apache Kafka kafka:9092 (In-cluster) / localhost:29092 (Host)
Frontend App	Vite Dev Server http://localhost:5173
Authentication	JWT HS256, Issuer: BuildNexusAuth, Audience: BuildNexusServices (60 min expiry)
Bootstrap Admin	admin@buildnexus.local / ChangeMe123! (Dev environment only via AdminSeeder)
Runtime Toolchain	.NET 10.0.400, Node 20.19.5 (QA Host) / Node 22 (CI), Docker Desktop

```
# Commands to start the test environment stack:
cd BuildNexus/infra && cp .env.example .env && docker compose up -d
cd ../frontend && npm run dev
```

4. QA APPROACH & STRATEGY

1. **Automated Backend Unit & Policy Tests:** Executed via xUnit (dotnet test) for isolated logic, claim checks, and controllers.
2. **Automated Database Integration Tests:** xUnit with real MySQL containers in GitHub Actions CI pipeline.
3. **Automated Frontend Component Tests:** Vitest + React Testing Library running in CI environment.
4. **Manual / Exploratory API Testing:** Verification using Postman Collection (BuildNexus_Sprint1.postman_collection.json) covering positive, negative, and unauthorized paths.
5. **End-to-End UI Verification:** Golden-path walkthroughs per user story against the integrated local Docker stack.
6. **Security & Resilience Auditing:** Token tampering, role escalation checks, self-lockout prevention, timing attack resistance, and SQL injection checks.

5. AUTOMATED TEST EXECUTION RESULTS

Test Suite	Passed	Failed	Not Run (Local)	Total	Execution Notes
api-gateway-tests	48	0	0	48	Routing, token validation, startup config
user-service-tests	141	0	30	171	30 WebApplicationFactory + MySQL integration tests require DB
project-service-tests	216	0	11	227	11 DB repository tests require infra

Test Suite	Passed	Failed	Not Run (Local)	Total	Execution Notes
Backend Total	405	0	41	446	100% pass rate when infra active / in CI
frontend (Vitest)	Passed in CI Environment			7 suites	Requires Node 22 (QA host Node 20 incompatible with jsdom)

Note on Local Test Run Execution: The 41 tests reported as "Not run locally" represent database-integration tests. They failed locally only when MySQL was off (Unable to connect to MySQL host). When running `docker compose up user-db project-db` or running inside the GitHub Actions CI pipeline, all 446 tests pass with 0 assertion failures.

Backend Coverage Breakdown by Area

Functional Testing Area	Representative Test Classes	Test Count
Registration Role Rules / Anti-Escalation	RegistrationRoleTests, RoleAccessTests	10
Password Reset Token Lifecycle	PasswordResetTokenServiceTests, AuthControllerPasswordResetTests	37
Profile Update Rules	UpdateProfileRequestTests, UsersControllerProfileTests	18
Admin User Directory & Filters	AdminUserDirectoryTests	15
Admin User Management & Self-Guards	AdminUserManagementTests, AdminUserRequestTests	42
Endpoint Role Declarations	EndpointRoleDeclarationTests	11
Migration Scripts	MigrationScriptTests	19
Create Project Validation	CreateProjectRequestTests	26
Project Access Policy	ProjectAccessPolicyTests	11
Project Status Transitions (State Machine)	ProjectStatusTransitionsTests, UpdateProjectStatusRequestTests	12
Project Status Endpoint	ProjectStatusEndpointTests, ProjectsControllerTests	34
Event Outbox & Dispatcher (US-22)	ProjectCreatedEventTests, OutboxDispatcherTests	50
Status History DB Ordering	ProjectRepositoryDatabaseTests	11
Gateway Routing & Token Validation	RoutingTests, TokenValidationTests	18

6. DETAILED STORY-BY-STORY VERIFICATION

US-01 — User Registration & Login (✅ PASS)

#	Test Scenario	Method	Result
1.1	Register Client with valid payload → 201, body echoes profile, no password	API + Auto	PASS
1.2	Register with <code>role: "Admin"</code> → 400 Validation Error	API + Auto	PASS
1.3	Register with <code>role: "admin"</code> (lowercase) → 400 Case-sensitive rejection	Auto	PASS

#	Test Scenario	Method	Result
1.4	Register duplicate email → 409 Email already registered	API	PASS
1.5	Register weak password (<8 chars) → 400 Bad Request	API	PASS
1.6	Register invalid email format → 400 Bad Request	API	PASS
1.7	Login with correct credentials → 200 OK + JWT Token	API + Auto	PASS
1.8	Decoded JWT carries sub claim, role claim, ~60 min exp	API (jwt.io)	PASS
1.9	Login wrong password → 401 "Invalid email or password."	API	PASS
1.10	Login unknown email → 401 Generic message (timing-safe hash)	API	PASS
1.11	Login deactivated account → 401 Unauthorized	API + Auto	PASS
1.12	Passwords hashed with PBKDF2-HMAC-SHA256, never logged	Code Review	PASS

US-02 — Manage User Profile (✓ PASS)

#	Test Scenario	Method	Result
2.1	GET /api/users/me with valid token → 200 Profile payload	API + Auto	PASS
2.2	GET /api/users/me without token → 401 Unauthorized	API + Auto	PASS
2.3	PUT /api/users/me update name, phone, address → 200 Saved	API + Auto	PASS
2.4	PUT /api/users/me with email in body → 400 "Email cannot be changed here"	Auto	PASS
2.5	PUT /api/users/me with role in body → 400 "Role cannot be changed here"	Auto	PASS
2.6	Clear optional fields (phone/address) → Persisted as NULL	Auto + DB	PASS
2.7	Phone < 7 digits → 400 Validation Error	Auto	PASS
2.8	Name length validation (<2 or >150 chars) → 400 Validation Error	Auto	PASS

US-03 — Enforce Role-Based Access Control (✓ PASS)

#	Test Scenario	Method	Result
3.1	Every endpoint decorated with explicit <code>[Authorize(Roles=...)]</code>	Auto	PASS
3.2	Client calls GET /api/users (Admin only) → 403 Forbidden	API + Auto	PASS
3.3	Client calls GET /api/users/directory → 403 Forbidden	API + Auto	PASS
3.4	Admin calls GET /api/users/directory → 403 Forbidden (Admin is not staff)	Auto	PASS
3.5	Tampered token signature → 401 Unauthorized	Auto	PASS
3.6	Expired token → 401 with WWW-Authenticate Bearer challenge	Auto	PASS
3.7	Unauthenticated request to protected route → 401 RFC 7807 Problem Details	API + Auto	PASS
3.8	Unmatched path at Gateway → Deny-by-default (401/404)	Auto	PASS
3.9	UI Route Guard: Client hitting /admin/users redirected	Auto + UI	PASS

#	Test Scenario	Method	Result
3.10	Unauthenticated UI access redirects to /login	UI	PASS

US-04 — Password Reset (✓ PASS)

#	Test Scenario	Method	Result
4.1	Forgot password with known email → 202 Accepted + Email in Mailpit	API + UI	PASS
4.2	Forgot password with unknown email → 202 Accepted (No email sent)	API + Auto	PASS
4.3	Response wording identical for known/unknown emails (Anti-enumeration)	Auto	PASS
4.4	POST /api/auth/reset-password with valid token → 200 Password Updated	API + Auto	PASS
4.5	Reuse same token → 400 "This password reset link is no longer valid"	API + Auto	PASS
4.6	Expired token (>30 mins) → 400 Link Invalid	Auto	PASS
4.7	Subsequent forgot password request invalidates previous token	Auto	PASS
4.8	Forgot password for deactivated account → 202 Accepted, no email sent	Auto	PASS
4.9	Attempt login with old password after reset → 401 Unauthorized	API + Auto	PASS
4.10	Reset token hashed in DB; raw token never logged	Code Review	PASS
4.11	Reset password with weak string → 400 Validation Error	Auto	PASS
4.12	Concurrent redemption of single token → First succeeds, second gets 400	Auto	PASS

US-05 — Create Construction Project (✓ PASS)

#	Test Scenario	Method	Result
5.1	Client POST /api/projects valid body → 201 Created, Status: Pending	API + Auto	PASS
5.2	Architect / Manager / Admin attempt project creation → 403 Forbidden	API + Auto	PASS
5.3	Missing bedrooms field → 400 "Number of bedrooms is required"	Auto	PASS
5.4	bedrooms: 0 accepted as valid integer	Auto	PASS
5.5	floors: 0 → 400 Bad Request (Minimum 1 floor)	Auto	PASS
5.6	Budget / Land size ≤ 0 → 400 Bad Request	Auto	PASS
5.7	Budget overflow beyond DECIMAL bounds → 400 Validation Error	Auto	PASS
5.8	Project name < 3 or > 150 chars → 400 Bad Request	Auto	PASS
5.9	otherRequirements > 2000 chars → 400; blank → stored NULL	Auto	PASS
5.10	Injected status or clientId in payload ignored (extracted from JWT)	Auto	PASS

#	Test Scenario	Method	Result
5.11	Atomic write: project row + status history + Outbox event created	Auto + DB	PASS

US-06 — View & Update Project Status (✓ PASS)

#	Test Scenario	Method	Result
6.1	GET /api/projects Client sees own projects; Admin sees all	API + Auto	PASS
6.2	GET /api/projects/{id} as owning Client → 200 with full history	API + Auto	PASS
6.3	GET /api/projects/{id} as unassigned staff → 403 Forbidden	API + Auto	PASS
6.4	GET /api/projects/{invalid-guid} → 404 Not Found	API + Auto	PASS
6.5	Architect updates status Pending → Designing → 200 OK	API + Auto	PASS
6.6	Skip step (Pending → DesignApproved) → 400 Validation Error	Auto	PASS
6.7	Backward transition (Designing → Pending) → 400 Bad Request	Auto	PASS
6.8	Transition from Completed status → 400 Terminal status reached	Auto	PASS
6.9	Same-status transition attempt → 400 Bad Request	Auto	PASS
6.10	Owning Client attempts status update → 403 Forbidden	API + Auto	PASS
6.11	Invalid status string → 400 "That is not a project status"	Auto	PASS
6.12	Concurrent status change conflict → 409 Conflict	Auto	PASS
6.13	Status audit details (changedByUserId) verified from token	Auto	PASS
6.14	History ordering remains stable for sub-second changes	Auto (CI)	PASS

US-22 — Project Event Integration (Outbox → Kafka) (✓ PASS)

#	Test Scenario	Method	Result
22.1	Create project → ProjectCreated event written to Outbox	Auto	PASS
22.2	Status update → ProjectUpdated (and ProjectApproved) events raised	Auto	PASS
22.3	Event payload built from committed domain changes	Auto	PASS
22.4	Atomic transaction: rollback drops both domain row and outbox event	Auto	PASS
22.5	Background dispatcher drains Outbox to Kafka in FIFO order	Auto	PASS
22.6	Kafka outage → Event retained in outbox, retry count incremented	Auto	PASS
22.7	API response time isolated from Kafka network delays	Code Review	PASS
22.8	GET /api/projects/{id}/events as Admin → 200 with delivery logs	API + Auto	PASS
22.9	GET /api/projects/{id}/events as non-Admin → 403 Forbidden	API + Auto	PASS

US-25 — API Gateway Routing & Integration (✓ PASS)

#	Test Scenario	Method	Result
25.1	Route table routes requests to user, project, design, construction services	Auto	PASS
25.2	/api/auth/** allows anonymous; all other routes enforce auth	Auto	PASS
25.3	Anonymous call to /api/projects → 401 Unauthorized at Gateway	Auto	PASS
25.4	Unmapped route /api/unknown → 404 Not Found (Not forwarded)	Auto	PASS
25.5	Tampered/expired token rejected with Problem Details	Auto	PASS
25.6	Gateway JWT config matches User Service parameters	Auto	PASS
25.7	CORS preflight from http://localhost:5173 permitted	Auto	PASS

US-35 — Configure CI/CD Pipeline (✓ PASS)

#	Test Scenario	Method	Result
35.1	Pipeline triggers on push to all branches and PRs to develop/main	File Review	PASS
35.2	Backend build compiles all services in Release mode	CI Execution	PASS
35.3	Frontend build executes typechecks and Vite bundling	CI Execution	PASS
35.4	Database integration tests provision MySQL container in CI	CI Execution	PASS
35.5	Frontend Vitest suite runs under Node 22 environment	CI Execution	PASS
35.6	Concurrency rules cancel outdated builds on same ref	File Review	PASS

US-37 — Admin: Manage Users & Roles (✓ PASS)

#	Test Scenario	Method	Result
37.1	GET /api/users?page=1&pageSize=10 → 200 Paged user list	API + Auto	PASS
37.2	GET /api/users?role=Architect → Filtered list and count	API + Auto	PASS
37.3	Admin updates user role/email → 200 OK	API + Auto	PASS
37.4	Admin self-role change attempt → 400 "You cannot change your own role"	API + Auto	PASS
37.5	Admin self-deactivation attempt → 400 "You cannot deactivate your own account"	API + Auto	PASS
37.6	Deactivated user login attempt → 401 Unauthorized	API + Auto	PASS

7. OBSERVATIONS & NON-BLOCKING FINDINGS

- 7.1 Bootstrap Admin Active Only in Development Environment (INFO):** AdminSeeder runs exclusively when ASPNETCORE_ENVIRONMENT=Development. Production deployment requires a documented initial admin creation script in Sprint 2/3.
- 7.2 Seeded Admin Credentials Committed (INFO):** ChangeMe123! is committed in source code for local dev convenience. Mandatory password rotation policy required prior to production deployment.

3. **7.3 Tooling Version Mismatch on QA Host (TOOLING):** QA host Node version (20.19.5) causes Vitest failure locally (`markAsUncleanable`). CI environment uses Node 22 where all frontend tests pass. Action: Upgrade QA host to Node 22.
4. **7.4 Local Integration Test Dependency (DOCUMENTATION):** Running `dotnet test` without starting Docker MySQL databases causes 41 integration test connection errors. Action: Update README documentation to clarify prerequisites.

8. DEFECT LOG

No open or blocking defects exist in the current Sprint 1 release candidate. Two defects were identified and resolved within the sprint:

Jira Ref	User Story	Defect Description	Fix Commit	Verification
SCRUM-19	US-06	Status history could sort out of order when timestamps matched precisely	b819bcc	VERIFIED PASS
SCRUM-14	US-01	Self-registration allowed requesting Admin role via API payload	4deef07	VERIFIED PASS

9. SECURITY REVIEW SUMMARY

Security Vector / Audit Area	Status	Verification Detail
JWT Signature Integrity	PASS	Tampered signatures fail at Gateway and microservices (401)
Token Expiry Enforcement	PASS	Expired tokens strictly rejected with WWW-Authenticate header
Privilege Escalation Barriers	PASS	Self-registration to Admin blocked; profile updates exclude role/email
Horizontal Access Control	PASS	Clients restricted exclusively to their own project resources
Admin Self-Lockout Protection	PASS	System prevents admin self-deactivation and self-demotion
User Enumeration Protection	PASS	Auth endpoints return constant messages and equalized response times
SQL Injection Vulnerabilities	PASS	All database queries utilize parameterized ADO.NET/ORM calls
Gateway Route Hardening	PASS	Deny-by-default stance enforced for unmapped routes

10. REQUIREMENTS TRACEABILITY MATRIX

Story	AC Verified	Automated Coverage	Manual / API Coverage	Verdict
US-01	5 / 5	Yes	Yes	PASS
US-02	4 / 4	Yes	Yes	PASS
US-03	5 / 5	Yes	Yes	PASS
US-04	6 / 6	Yes	Yes	PASS
US-05	6 / 6	Yes	Yes	PASS
US-06	7 / 7	Yes	Yes	PASS
US-22	5 / 5	Yes	Yes	PASS

Story	AC Verified	Automated Coverage	Manual / API Coverage	Verdict
US-25	5 / 5	Yes	Yes	PASS
US-35	5 / 5	Yes	Yes	PASS
US-37	3 / 3	Yes	Yes	PASS

11. QA SIGN-OFF & APPROVAL

QA Lead Approval Status: APPROVED Build branch <code>feature/SCRUM-50</code> meets all functional, security, and integration criteria for Sprint 1 release. Signature / Date: 2026-09-03	Product Owner Sign-off Status: PENDING SIGN-OFF Accepted stories: US-01, US-02, US-03, US-04, US-05, US-06, US-22, US-25, US-35, US-37. Signature / Date:
---	--

12. REPORT ATTACHMENTS & ARTIFACTS

- `QA_SPRINT1_REPORT.md` — Markdown source document
- `BuildNexus_Sprint1.postman_collection.json` — Complete API integration regression test collection
- `BuildNexus_Local.postman_environment.json` — Local environment configuration for Postman